To make the rows hidden, tables get a deleted_at timestamp column that's NULL by default and then is set when a row is deleted. Any rows with a deleted_at value set are hidden.

Imagine that Users can deactivate their accounts and this action populates the deleted_at column value. The Marketing department wants to find those users and offer them incentives to come back to the platform.

Create the index again like you did earlier, except this time exclude the deleted users from the index.

Rideshare does not currently have Soft Deletes enabled, so you'll need to add the deleted_at column.

Run the following statements in psql for Rideshare to add the column, and then add a Partial Index. After that you'll delete about 10 users, and then analyze the table.

sql/index_users_deleted_email_partial.sql

```
ALTER TABLE users ADD COLUMN deleted_at timestamptz;

-- Index on deleted users
CREATE INDEX IF NOT EXISTS index_users_deleted_email_partial
ON users
USING btree (email)
WHERE deleted_at IS NOT NULL;

-- delete some users
UPDATE users
SET deleted_at = NOW()
WHERE id <= 10;

ANALYZE users;
```

Now try running this query:

```
SELECT email FROM users
WHERE deleted_at IS NOT NULL;
```

Prepend EXPLAIN ANALYZE to the query and confirm the Index is being used.

An alternative would be to use a Multicolumn index. Add an index covering the deleted_at column to filter down to only deleted rows, and then include the email column as the second column in the definition, so that the email can be fetched from the index without needing a Heap Scan.

Run this in psql:

```
CREATE INDEX IF NOT EXISTS users_deleted_email
ON users USING btree(deleted_at, email);
```